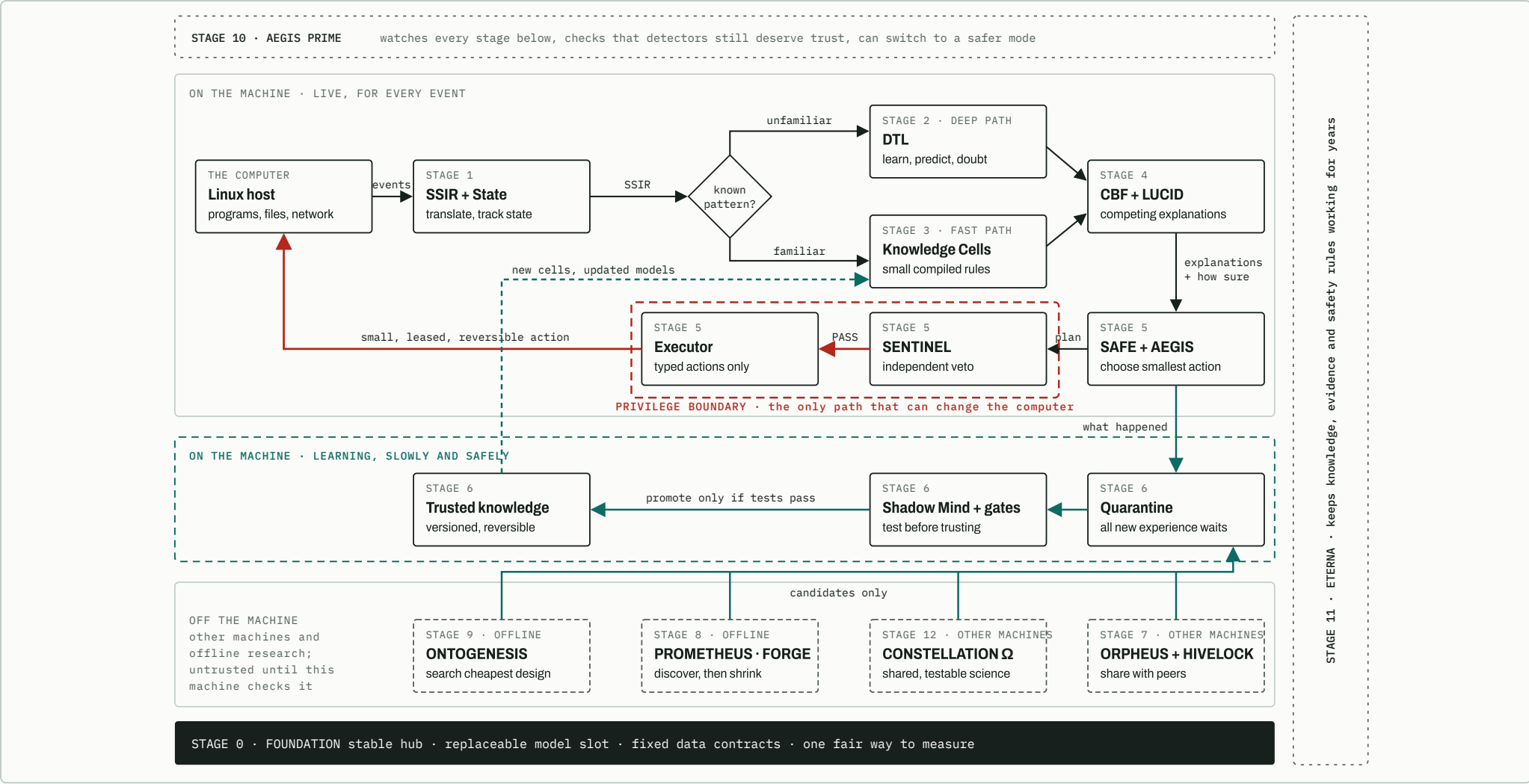


# PocketSec

PocketSec is a security system for small Linux machines with about 2 GB of memory. It watches what programs do, works out whether something dangerous is happening, and responds only in small, safe, reversible ways. Stages 1 to 5 form the live pipeline. Stage 1 translates raw system events into a short security language and keeps track of what is now true about the machine. At runtime, cheap compiled rules from Stage 3 handle routine events first, and only unfamiliar ones wake the Stage 2 learning model. Stage 4 weighs competing explanations and says 'unknown' when the evidence cannot decide, and Stage 5 chooses the smallest safe action, which must pass an independent checker. Stages 6 to 12 wrap around this pipeline: they learn from outcomes through a quarantine (6), share knowledge with other machines (7 and 12), research unexplained behaviour and smaller designs offline (8 and 9), keep checking that detectors still deserve trust (10), and keep the system trustworthy as hardware and software change over the years (11). None of these outer stages can change the live system or trigger an action directly. Every stage is a hypothesis that must beat a simpler method on the same data and hardware.

|          |                                                                                                           |
|----------|-----------------------------------------------------------------------------------------------------------|
| PROTECTS | One small Linux computer with about 2 GB of memory                                                        |
| WATCHES  | What programs do: start, read, write, connect, gain privilege                                             |
| DECIDES  | Is this dangerous? How sure are we? What is the smallest safe response?                                   |
| BUILT AS | 13 stages (0–12), each a separate layer with fixed inputs and outputs                                     |
| KEY IDEA | Routine behaviour becomes cheap rules; expensive thinking is kept for what is new, uncertain or dangerous |

# The big picture



Read the top region left to right, then the second row right to left: an event is translated (1), handled cheaply if familiar (3) or examined by the learning model if not (2), explained (4), and answered (5). Only the executor, inside the red boundary, can touch the computer. Below that, all learning goes through one quarantine (6), and everything from other machines or offline research must enter through that same door.

# Follow one attack through the system

One story, stage by stage. Steps 1–6 happen on the machine while the attack is running. Steps 7–12 happen afterwards, over days to years.

**The attack.** On a small Linux server, the web server is tricked into starting a shell. The shell drops a hidden program, runs it, and the program reads the password file and then calls out to another computer.

web server → shell → writes /tmp/.x91 → runs it → reads /etc/shadow → connects to 10.0.0.9

---

|                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1</b><br>STAGE 1 · SSIR +<br>SECURITY STATE<br><b>The event is translated</b> | <p>The Semantic Compiler turns the incident into a chain of SSIR transitions: web server SPAWN shell, shell CREATE/WRITE /tmp/.x91, EXECUTE /tmp/.x91, new process READ /etc/shadow (a FILE with the CREDENTIAL property), then CONNECT to ENDPOINT 10.0.0.9. Each transition has an evidence pointer and a causal signature that links it back to the web server. /tmp/.x91 starts as an UNKNOWN executable and earns meanings such as CREDENTIAL_READER and NETWORK_CLIENT from what it does. Because /etc/shadow is normally readable only by root, the Host Security State for this chain shows privilege at root, credential exposure rising from none to readable, and reachability rising to LAN (10.0.0.9 is a private address). <math>\Phi</math> rises because of this combination. The chain is high in novelty, potential and uncertainty, so it is kept at a rich representation level and never aggregated away, and AOP may raise telemetry detail on it within its caps.</p> |
| <b>2</b><br>STAGE 3 · AICT +<br>CRYSTAL<br><b>Is it a known pattern?</b>         | <p>When the web server spawns the shell and the new process reads /etc/shadow, the Boundary Index looks for a validated cell; a cell such as NETWORK_SERVICE -&gt; CHILD_INTERPRETER -&gt; CREDENTIAL_EXPOSURE could fire cheaply to raise risk, require evidence to be kept and request more observation, but if no cell covers this new chain, or it sits near a boundary, the event goes to Stage 2 DTL instead. Meanwhile the web server's ordinary request handling is answered by cheap cells, so DTL can focus on this incident, and Stage 3 will never crystallize this chain as 'normal' just because it repeats.</p>                                                                                                                                                                                                                                                                                                                                                               |

---

|   |                                   |                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---|-----------------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3 | STAGE 2 · DTL                     | <b>The learning model looks closer</b> | <p>The shell started by the web server, the run of the new file /tmp/.x91 and the read of /etc/shadow are all novel and high in security potential. The Need-to-Compute Gate therefore sends them up to the deep paths (P3/P4). The Future Cone shifts probability toward a 'credential-to-egress chain' branch, with high relation, object and state surprise. Uncertainty and risk are both high, so DTL asks Stage 1's Adaptive Observation Policy for richer telemetry and preserves evidence. The Counterfactual Twin and Causal Credit Ledger mark the web-server-to-shell step and the /etc/shadow read as the steps most responsible before the 10.0.0.9 connection. The anti-poisoning gate keeps these samples in quarantine as evidence, so repeats of the chain are not quietly learned as normal behaviour.</p>                                             |
| 4 | STAGE 4 · CBF + LUCID             | <b>Which explanation is true?</b>      | <p>LUCID keeps several competing worlds alive for this incident. One: the web server was exploited, and an attacker is taking credentials and calling out. Two: legitimate automation that uses an unusual destination. Three: an UNKNOWN new mechanism. The /etc/shadow read is recorded only as [OBS] a read of credential material, which supports [INF] possible credential access. To separate the worlds cheaply, LUCID may ask Stage 1 for one targeted observation, such as the hash of /tmp/.x91 or a short trace of file access by that process and its children. It then hands Stage 5 the surviving worlds, with an explicit [UNK] for direct exfiltration.</p>                                                                                                                                                                                              |
| 5 | STAGE 5 · SAFE + AEGIS + SENTINEL | <b>Decide the smallest safe action</b> | <p>Given Stage 4's worlds, AEGIS first chooses low-risk steps. It preserves evidence (a snapshot of the incident metadata and the state of the /tmp/.x91 process). It then proposes a temporary, leased restriction on that one process's connection to 10.0.0.9, or suspending the process with a validated resume. It does not contain the web server itself, because service containment (O5) needs a human by default, and if the web server is listed as a critical service in the mission invariants it may not be interrupted autonomously at all. SENTINEL checks the action against scope, authority and the mission invariants. Before acting, the executor re-checks the target's identity (PID, start time, executable hash). Afterwards it verifies that the connection was actually blocked, and if it was not, it rolls back or escalates to a human.</p> |

## 6

STAGE 6 · HELIOS +  
MNEMOSYNE

### Learn from what happened

Stage 6 does not act during the incident. Afterwards the resolved chain (web server spawns a shell, hidden file /tmp/.x91 is written and run, /etc/shadow is read, connection to 10.0.0.9) and Stage 5's verified outcome become an ExperienceCapsule in quarantine; because reading password hashes touches the protected credential-material meaning, Semantic Homeostasis ensures that even if an attacker repeats this chain many times it can never be learned as normal. HELIOS may propose a candidate Knowledge Cell for the chain, which is rehearsed with renamed files and shifted timing, run in the Shadow Mind and passed through the Conservation Gate and a canary before it becomes trusted, with a fossil kept for rollback.

## 7

STAGE 7 · ORPHEUS +  
HIVELOCK

### Tell other machines, carefully

Stage 7 does not see this incident live. Once resolved, the Antibody Forge can turn it into a signed ANTIBODY capsule in which exact details such as /tmp/.x91 and 10.0.0.9 are replaced by general classes (a hidden temporary executable, an outbound destination) and no password material is ever exported; other hosts treat it only as a candidate that must pass HIVELOCK and their own Stage 6 checks. If other hosts report weak fragments of the same pattern, such as destination novelty, the Partial-World Reconstructor may propose a cross-host campaign, which the Consensus Falsifier tests against coincidence or a shared benign software update before it counts.

## 8

STAGE 8 · PROMETHEUS +  
ORACLE + FORGE

### Research what stayed unexplained

Stage 8 would not act on this incident live. If incidents like it (web server spawns a shell, a hidden /tmp file is run, /etc/shadow is read, an outbound connection follows) keep leaving residuals, PROMETHEUS would propose a typed mechanism such as execution → file write → execution → sensitive read → new destination, and set it against innocent 'doppelgängers' like an admin or backup script. ORACLE would then test it on replayed history, for example by renaming /tmp/.x91 or changing the timing, to confirm the theory depends on behaviour and not on names. FORGE would compile the survivor into a tiny finite-state-machine detector and send it to Stage 6 quarantine as a candidate. The novelty audit would probably label it known ATT&CK behaviour rather than a new discovery.

## 9 STAGE 9 · ONTOGENESIS

### Look for a cheaper design

Stage 9 does not handle this incident live. It uses replays of incidents like it to find the smallest computation that still catches the whole chain. For example, the Symmetry Laboratory would learn that the exact name `/tmp/.x91` and the PIDs do not matter, while the privilege-sensitive read of `/etc/shadow` followed by a new outbound destination does. CHRONOS would decide how long the 'web server spawned a shell' fact must be remembered so that the later read and connection can be linked to it. The winning design, for instance a small state machine for the web-server phenotype, would go to Stage 6 as a `SuccessorPackage` only after ARGUS fails to break it.

---

## 10 STAGE 10 · AEGIS PRIME

### Keep checking the detectors

When the detector flags the chain (web server spawns a shell, the shell writes and runs `/tmp/.x91`, that process reads `/etc/shadow` and connects to `10.0.0.9`), Stage 10 makes the alert carry an evidence certificate. The certificate types each link, for example the shadow read as `DIRECT` and the attack mechanism as `INFERRED`, and names the smallest witness, such as the `/etc/shadow` read followed by the outbound connection. If the network sensor was down, the connection is marked `MISSING` rather than 'did not happen', and the alert's confidence is capped. SENTINEL confirms that the evidence exists and that the host is inside its validated envelope, JANUS may re-check this high-value episode with a different detector type, and the prediction is written to the immutable ledger so that it can be scored against the investigation's outcome later.

---

## 11 STAGE 11 · ETERNA

### Stay trustworthy for years

Stage 11 does not judge this single incident live. The incident context can, however, make HIBERNIA verify (through ANCHOR's signature) and load a deeper credential-theft specialist from the signed cold store into RAM. Afterwards, knowledge from incidents like this is kept as a historical incident fossil and an ancestral probe, so that any future detector generation (after a kernel upgrade, a sensor replacement or a move to ARM hardware) must still catch 'web shell drops a `/tmp` file that reads `/etc/shadow` and connects out' before it is promoted. PALIMPSEST keeps the raw telemetry only for a short, bounded time, while the validated mechanism is kept long term.

---

**Build shared knowledge**

Stage 12 does not stop this incident live; the local Stage 1-11 pipeline does. Afterwards, VEIL could turn it into an abstract atom such as 'web-server process -> shell -> hidden file written in /tmp and run -> password-hash file read -> outbound connection', stripping identifiers like the exact file name, the address 10.0.0.9 and precise timestamps, and attaching tests and counterexamples. Other nodes would quarantine that atom, reproduce it in SANCTUM, recompile it with GENESIS and adopt it only after their own checks; they would never block anything simply because a peer said so.

---

# The 13 stages in detail

Each stage below explains the problem it solves, what goes in and out, its main parts, how it works, and the rules it must never break.

---

## The foundation

Rules and data formats every other stage must follow.

### 0

## Foundation

Fundamental Architecture Discovery · NERA research foundation · Rules for all stages

Stage 0 sets the ground rules for the whole project: the scientific question, the fixed boundary between the long-lived product and its swappable AI part, the data formats between them, and the rules for measuring any idea fairly.

NERA = Novelty-Energy Relational Automaton · JIT = just-in-time (compilation) · ADR = Architecture Decision Record

#### THE PROBLEM IT SOLVES

Security tools on small machines usually run the same expensive analysis on every event, even boring routine ones, and they get tied to one kind of AI model. Stage 0 asks whether routine behaviour can become cheap to check, so heavy computation is spent only on what is new. It also blocks ideas that only sound clever by requiring every mechanism to beat simple baselines under the same data and hardware.

#### THINK OF IT AS

*It is like the building code and foundation of a house: it fixes where the load-bearing walls go and how every room will be inspected, and it leaves the furniture (the AI model) free to be replaced later.*

#### WHAT GOES IN

- The central research question: can learned Linux security intelligence compile itself into a small executable behaviour machine, so heavy neural computation is needed mainly for novelty?
- Linux telemetry, which the hub's normalizer turns into security events (SecurityEventV1) and passes to the model slot as a SecurityEventSequenceV1; Stage 0 fixes only the contract, and Stage 1 researches the exact fields
- The fixed hardware and resource profiles (Nano, Edge, Research Max) that every design must be measured against

#### WHAT COMES OUT

- ← Versioned contracts
  - SecurityEventSequenceV1 (hub to model slot) and ThreatPredictionV1 (model slot back to hub); Stage 0 fixes only their shape, and Stage 1's SSIR transitions and

Host Security State become the actual content that flows into the model slot

- ← A benchmark harness and measurement framework that always reports security quality and resource cost together
- ← An experiment registry with fixed experiment IDs, a reproducibility policy, an ADR template and a prior-art ledger
- ← Formal Stage 1 entry criteria: Stage 1 then researches the smallest useful event representation

## MAIN PARTS

### Stable PocketSec Hub

The long-lived part of the product (telemetry, normalizer, deterministic rules, correlation, risk, evidence and the command-line/alert interface) that must not depend on any one model family.

### ThreatPredictionV1 contract

The fixed answer format from the model: a threat/state identifier, a calibrated confidence, a novelty score, references to the evidence, a model version and an explicit 'I am not sure' (abstention) signal.

### JIT Intelligence / Knowledge Compilation

When the learner finds a stable, validated pattern, it is turned into a cheap executable detector or state transition; if that detector starts to fail, it is demoted and its examples go back to the learner.

### Model Slot

A replaceable socket where the learned intelligence plugs in; it takes a SecurityEventSequenceV1 and must return a ThreatPredictionV1, so models can be swapped without changing the hub.

### Novelty-Energy Principle

Measure how surprising each event is given the current state, and scale the compute budget with that surprise: known events take a cheap table lookup, unusual ones a small scorer, novel high-risk ones the deep solver.

### Hierarchical Behaviour Memory

A four-tier memory (HOT exact recent state, WARM compressed transitions, COLD fingerprints, FORGOTTEN removed) that keeps memory bounded while still recognising old behaviour.

## HOW IT WORKS

1. Stage 0 freezes one optimisation goal: the smallest model that keeps the required security quality, while also keeping model size, compute per event and storage cost low.

## RULES IT NEVER BREAKS

- The hub must never depend on one model family; the AI part stays replaceable behind versioned contracts.

2. It draws a hard line between the stable hub and the replaceable Model Slot, and fixes the two versioned data formats that cross that line.
3. At runtime each event is first checked against known transitions; if it is known, a cheap state transition handles it, and only if it is not known does the novelty budget wake the learned solver.
4. Patterns the learner discovers are validated, minimised and compiled into cheap detectors (the Learned Behaviour Machine of states and transitions); failing detectors are demoted back to learning.
5. It lists competing hypotheses (H0 to H8, from a plain compact classifier up to the combined NERA design) as things to test, not results to claim.
6. Every later experiment must go through the same benchmark harness with the same data split, hardware and budgets, reporting security metrics next to RAM, CPU and latency.

- Do not learn what deterministic code, rules, metadata or external knowledge can represent exactly.
- Compiled knowledge must be reversible: a degrading detector can be demoted back to the learner.
- No component survives because it sounds advanced; it must beat or complement strong conventional baselines under identical conditions, and synthetic-data results are never presented as real-world detection quality.

#### WHERE IT RUNS

Mostly design-time: the contracts and the hub/model-slot boundary are built into the agent on the small endpoint (Edge target about 100 MB RAM for the agent), while the benchmark harness, experiment registry and hypothesis testing run offline on a research machine.

#### IN THE EXAMPLE ATTACK

Stage 0 does not analyse the incident itself; it fixes the shape everything travels in. The web-server-to-shell chain reaches the Model Slot as a `SecurityEventSequenceV1`, and the answer must come back as a `ThreatPredictionV1` with confidence, a novelty score, evidence references and the option to abstain. Because the chain is surprising (a web server does not normally spawn a shell that runs a new file and reads `/etc/shadow`), the Novelty-Energy Principle sends it to the expensive path; if chains like this are later learned and validated, JIT compilation could turn them into a cheap transition, like the doc's example `'credential_read then unknown_external_network leads to RISK_ESCALATION'`.

---

# On the machine: understanding what is happening

Runs live on every event. At runtime the order is 1 → 3 → 2 → 4: Stage 3's cheap rules handle what is already understood, and only the rest reaches Stage 2.

## 1 SSIR + Security State

Security Reality Encoding and Adaptive Observation · Runs on the 2 GB machine

Stage 1 turns messy raw Linux logs into a short, standard 'security language' of what changed, keeps a running picture of what is now true about the machine, and spends extra observation only where doubt and danger justify it.

SSIR = Security Semantic Intermediate Representation · AOP = Adaptive Observation Policy

### THE PROBLEM IT SOLVES

Different Linux sensors describe the same action in different, noisy ways, and raw logs are too large and too name-based for a small machine to reason over. A single log line also does not say whether the machine is now more dangerous, for example whether a process has gained root, read a secret and reached the network. Stage 1 builds a compact, sensor-independent representation that keeps what matters for security and keeps the raw evidence separately so nothing needed for investigation is lost.

### THINK OF IT AS

*It is like a hospital triage nurse who turns a patient's rambling story into a standard chart of symptoms, keeps an up-to-date summary of the patient's condition, files the original notes for the doctor, and orders extra tests only for patients who are both unclear and potentially serious.*

### WHAT GOES IN

- Raw Linux telemetry from sources such as eBPF, LSM, tracepoints, audit, journald and procfs
- Host metadata (for example file ownership and which files are credential files) used to give objects their meaning
- Evidence of legitimate system change (kernel build, package set, enabled services, config) used to decide behaviour epochs
- Requests from Stage 2 (high uncertainty) and Stage 4 (one targeted observation to tell worlds apart) asking the Adaptive Observation Policy to raise telemetry detail for a short, capped time

### WHAT COMES OUT

- ← SSIR transitions: compact records of who did what to what, with the security change, uncertainty, novelty, causal ancestry, timing and an evidence pointer; these feed Stage 2's predictive learning
- ← The Host Security State (what is now true: privilege, credential exposure, reachability, persistence and more) plus the Security Potential  $\Phi$

- ← Links into the Evidence Store, so the original raw record is always recoverable for investigation
- ← Stage 2 model-facing interface fixtures and a replay corpus
- ← The current Behaviour Epoch (which configuration period the machine is in) and the compressed causal state (the chain of responsible ancestors), each with its uncertainty, handed to Stage 2 together with the SSIR transitions

## MAIN PARTS

### Semantic Compiler

The compatibility boundary: it assembles raw events, then its entity, relation, state-delta and evidence-linker parts turn them into SSIR transitions, so different sensors describing the same action produce the same result.

### Host Security State and Security State Calculus

A compact running summary of what is now true about the machine, stored as ordered levels (for example Privilege user < elevated < root, Reachability none < local < LAN < external) that each transition can move up or down.

### Conditional Novelty Tensor

Measures how unfamiliar a transition is in several contexts at once (host, user, actor, parent-child pair, object, relation, time, causal chain) using bounded memory structures like Count-Min Sketch and Bloom filters.

### SSIR transition $\tau = (A, R, O, \Delta S, U, N, P, T, E)$

The basic unit of meaning: an Actor does a Relation (a small fixed set like SPAWN, EXECUTE, READ, WRITE, CONNECT) to an Object, plus the security change, uncertainty, novelty, provenance, timing and an Evidence reference.

### Security Potential $\Phi(S)$

A measure of how dangerous the current combination of capabilities is, because individually normal abilities (root, reading secrets, network reach) can be dangerous together.

### Adaptive Observation Policy (AOP)

Temporarily turns up optional, higher-resolution telemetry where uncertainty, security potential and causal relevance are all high, within strict CPU, memory, rate and duration caps.

## HOW IT WORKS

1. The Raw Event Assembler fuses compound telemetry records that describe one logical operation into a single raw event, keeping it losslessly as evidence.

## RULES IT NEVER BREAKS

- Raw evidence and model representation are separate: compression must never destroy the investigator's evidence trail,

2. The Semantic Compiler gives each entity three layers (exact identity, semantic meaning, evidence pointer) and maps the operation to a relation, producing an SSIR transition; unknown programs start as UNKNOWN and earn meaning from their behaviour, not their name.
3. Each transition updates the Host Security State ( $S_{next} = T(S, \tau)$ ), and the change in Security Potential  $\Delta\Phi$  shows whether the machine became more dangerous; causal memory keeps the chain of responsible ancestors at several resolutions without an unbounded graph.
4. Three signals stay separate: novelty (how unfamiliar), potential  $\Phi$  (how sensitive the resulting state is) and confidence/uncertainty (how sure the interpretation is); behaviour epochs let 'normal' shift after proven legitimate system changes.
5. Richness is escalated only when needed (levels L0 to L3), repetitive low-risk events are aggregated, and high-consequence transitions such as privilege, credential, persistence or boundary-crossing are never hidden.
6. Offline, the Information Guillotine removes information families one by one to find the smallest representation that keeps security performance, so every retained field has measured justification.

and no generative model participates in evidence creation.

- Unknown or incomplete semantics increase uncertainty; they never default to benign or malicious, and rare is never treated as malicious.
- All queues and caches are bounded, and adaptive observation has rate, duration and memory caps and never silently disables mandatory deterministic security signals.
- Epoch changes require corroborating system-change evidence and cannot be triggered by behavioural novelty alone (this resists attackers slowly poisoning the baseline).

#### WHERE IT RUNS

On the endpoint (the small 2 GB machine) as the live encoding pipeline with hard memory caps; the Information Guillotine and adversarial representation tests are research experiments run offline.

### IN THE EXAMPLE ATTACK

The Semantic Compiler turns the incident into a chain of SSIR transitions: web server SPAWN shell, shell CREATE/WRITE /tmp/.x91, EXECUTE /tmp/.x91, new process READ /etc/shadow (a FILE with the CREDENTIAL property), then CONNECT to ENDPOINT 10.0.0.9. Each transition has an evidence pointer and a causal signature that links it back to the web server. /tmp/.x91 starts as an UNKNOWN executable and earns meanings such as CREDENTIAL\_READER and NETWORK\_CLIENT from what it does. Because /etc/shadow is normally readable only by root, the Host Security State for this chain shows privilege at root, credential exposure rising from none to readable, and reachability rising to LAN (10.0.0.9 is a private address).  $\Phi$  rises because of this combination. The chain is high in novelty, potential and uncertainty, so it is kept at a rich representation level and never aggregated away, and AOP may raise telemetry detail on it within its caps.

## 2 DTL

Dynamic Transition Lattice intelligence core · Runs on the 2 GB machine

Stage 2 is PocketSec's first learning brain: it learns how a Linux machine normally behaves, predicts what is likely to happen next, and says honestly when it does not know.

DTL = Dynamic Transition Lattice · AOP = Adaptive Observation Policy (the Stage 1 component that decides how much telemetry to collect)

### THE PROBLEM IT SOLVES

Stage 1 describes what is happening on the machine, but describing is not the same as anticipating. A simple classifier only gives one label per event, and large AI models are far too heavy for a 2 GB machine. PocketSec needs a small learning core that predicts security-relevant futures, measures its own uncertainty, and keeps most of its expensive machinery asleep most of the time.

### THINK OF IT AS

*It works like an experienced weather forecaster who gives chances for several possible tomorrows, spends effort only on unusual days, and says 'I am not sure' when the pattern is new.*

### WHAT GOES IN

- SSIR transitions from Stage 1 (semantic descriptions of each event, such as 'a service started an interpreter')
- Host Security State from Stage 1 (what privileges, credentials and network exposure currently exist)
- Behaviour Epoch (the current configuration regime of the machine, e.g. before or after a package upgrade)
- Causal state and uncertainty carried forward from Stage 1

### WHAT COMES OUT

- ← Evidence-bound predictions (Future Cone, security-state change, hazard, surprise vector, uncertainty) that always point

back to immutable Stage 1 evidence IDs, for later decision stages

- ← Requests to Stage 1's Adaptive Observation Policy (AOP) for richer telemetry when uncertainty is high
- ← Validated Behaviour Atoms, transition statistics, uncertainty envelopes, causal evidence requirements and compile candidates handed to Stage 3
- ← Abstention ('unknown') and a fallback to Stage 1 deterministic rules if the model fails

## MAIN PARTS

### **Need-to-Compute Gate (Relevance / Need Router)**

A tiny router that scores each event for novelty, security potential, uncertainty, causal responsibility and prediction error, then picks the cheapest safe path, from P0 (a cached answer) up to P4 (deep counterfactual analysis plus a request for more telemetry).

### **Behaviour Atoms and the Dynamic Transition Lattice**

Recurring behaviour is snapped to reusable discrete prototypes (atoms) linked by probabilistic transitions; atoms with equivalent futures merge, and atoms that mix incompatible futures split (fission).

### **Counterfactual Twin and Causal Credit Ledger**

The Twin asks 'what would the predicted future be if this step had not happened?' to measure each step's responsibility; the Ledger keeps a bounded list of the steps that really changed the predicted security outcome, so an analyst sees a short causal story.

### **Multi-timescale state $z_t$**

A compact memory split into blocks for different time spans (fast, session, host, epoch, causal, uncertainty); only the blocks an event concerns are updated, the rest simply fade.

### **Future Cone and predictive heads**

Eight small prediction heads estimate a short, bounded set of plausible futures with probabilities (for example 'normal continuation' versus 'credential-to-egress chain') and a surprise vector that shows in which direction an event is unusual, not just how unusual.

### **Anti-Poisoning Learning Gate**

New experience goes into a quarantine buffer and passes consistency, epoch and risk checks with delayed promotion before it can change trusted state; suspicious samples are kept as evidence, not learned as normal.

## HOW IT WORKS

1. The Semantic Event Encoder turns each Stage 1 SSIR transition into a compact form, and the Need-to-Compute Gate decides how much thinking the event deserves (paths P0 to P4).
2. Only the relevant blocks of the multi-timescale state are updated; the others decay, so one event does not wake the whole model.
3. The continuous state is mapped to the nearest Behaviour Atom in the Dynamic Transition Lattice, which records which atoms usually follow which, with probabilities tied to the current epoch.
4. The predictive heads produce a Future Cone of a few possible futures, a security hazard estimate, and a structured surprise vector; missing expected events ('negative space') can also count as surprise.
5. Uncertainty is estimated separately: low uncertainty and low risk take the cheap path, while high uncertainty and high risk trigger an AOP request for richer telemetry, deeper inference and evidence preservation; the Counterfactual Twin and Causal Credit Ledger identify which steps mattered.
6. Transitions that stay stable and well calibrated become compile candidates for Stage 3; if drift or error rises, knowledge moves back from executable to candidate to discrete to neural form.

## RULES IT NEVER BREAKS

- Prediction failure never suppresses deterministic Stage 1 evidence; every alert references immutable evidence IDs, and generated explanations cannot create evidence.
- Unknown, NaN or corrupt model state causes abstention and a safe state reset, never a 'benign' classification.
- The model must not 'learn away' malicious behaviour merely because it repeats; compile candidates never become trusted solely from frequency.
- All dynamic structures (atoms, lattice, caches, adaptation buffers) have hard caps and deterministic eviction, within the 2 GB Edge resource budget (initial ceiling 80 MB extra RAM).

## WHERE IT RUNS

On the endpoint (the small 2 GB machine) for inference, bounded statistics and carefully gated adaptation; training happens on a separate development machine.

#### IN THE EXAMPLE ATTACK

The shell started by the web server, the run of the new file /tmp/.x91 and the read of /etc/shadow are all novel and high in security potential. The Need-to-Compute Gate therefore sends them up to the deep paths (P3/P4). The Future Cone shifts probability toward a 'credential-to-egress chain' branch, with high relation, object and state surprise. Uncertainty and risk are both high, so DTL asks Stage 1's Adaptive Observation Policy for richer telemetry and preserves evidence. The Counterfactual Twin and Causal Credit Ledger mark the web-server-to-shell step and the /etc/shadow read as the steps most responsible before the 10.0.0.9 connection. The anti-poisoning gate keeps these samples in quarantine as evidence, so repeats of the chain are not quietly learned as normal behaviour.

## 3 AICT + CRYSTAL

Adaptive Intelligence Crystallization · Runs on the 2 GB machine

Stage 3 turns behaviour that Stage 2 already understands well into small, cheap, rule-like Knowledge Cells, and 'melts' them back into learning when the machine changes.

AICT = Adaptive Intelligence Crystallization Theory · PCB = PocketSec Cell Bytecode · DTL = Dynamic Transition Lattice (Stage 2, used here as the teacher)

#### THE PROBLEM IT SOLVES

Running the learning model on every event costs CPU and memory, yet much of a server's behaviour is repetitive and well understood. Stage 3 converts that settled knowledge into cheap executable form so the model can sleep. A rule without limits silently extrapolates and reality changes, so every cell must carry its validity boundary and must be reversible.

#### THINK OF IT AS

*It works like learning to drive: gear changes that once needed full attention become reflexes, but when you drive abroad on the other side of the road you think carefully again, only about the parts that changed.*

#### WHAT GOES IN

- From Stage 2: validated Behaviour Atoms, transition statistics, uncertainty envelopes, causal evidence requirements and compile candidates
- Normalized Stage 1 SSIR events and security state at runtime (the only data the Cell VM may read)
- Explicit Stage 1/3 security invariants and labelled ground truth where available (the second oracle)
- The Stage 2 DTL model as a 'teacher' for comparison, shadowing and sampled audits
- Feedback from later stages: stress signals and candidate incident patterns from Stage 4, the Intervention Residual from Stage 5, and Knowledge Cells promoted by Stage 6 learning

## WHAT COMES OUT

- ← Knowledge Cell results: security-state updates, risk/hazard outputs, evidence requirements and AOP (observation escalation) requests, passed to Stage 4 through a frozen interface
- ← Events with no cell, conflicting cells or uncertainty, sent back to Stage 2 DTL
- ← Melted (no longer trusted) regions returned to DTL learning, and counterexamples used to improve DTL training
- ← Assurance records (certificate levels A0-A5) for each cell

## MAIN PARTS

### Knowledge Cell

A small executable unit that stores a learned relationship (invariant), where it is allowed to operate (boundary), the cheap operator that runs it, its confidence, evidence lineage, security constraints, valid epochs, audit policy and version.

### Knowledge Boundary and Boundary Pressure

Every cell must state where it is valid; Boundary Pressure deliberately nudges test cases (timing, actor, privilege, epoch, missing evidence) until the cell's answer diverges, to find where that edge really lies.

### PocketSec Cell Bytecode (PCB) and Cell VM

A tiny instruction set with no loops, recursion, dynamic memory or system calls, checked by a verifier and run in an unprivileged virtual machine that only sees normalized Stage 1 data.

### Behavioural Invariant discovery

Finds a general pattern behind specific cases, for example turning 'nginx / apache / custom service starts an interpreter that reads credentials' into NETWORK\_SERVICE -> CHILD\_INTERPRETER -> CREDENTIAL\_EXPOSURE.

### Dual Oracle

A candidate cell passes only if it stays close to what the DTL teacher predicts AND satisfies explicit hard security rules, so the teacher's own mistakes are not frozen into cells.

### Knowledge Field, Boundary Index and melting

All cells form a bounded field; the Boundary Index finds which cells apply to an event, a Cell Resolver combines them under fixed conflict rules, and cells under stress melt partly or fully back into DTL learning.

## HOW IT WORKS

1. At runtime the order differs from the numbering. Each Stage 1 event first goes to the Boundary Index. If a validated cell matches, it runs in the Cell VM. Only events with no cell, a conflict or uncertainty go on to Stage 2 DTL. DTL also audits a sample of cell decisions, at a rate set by consequence and by how close the event is to the cell's boundary.
2. A region miner picks areas of stable DTL behaviour worth compiling using Knowledge Pressure (how frequent, how costly, how predictable and how security-relevant), keeping compute-saving and security priority separate.
3. CRYSTAL discovers a Behavioural Invariant and a validity boundary; the evidence required (Resolution) rises with the security consequence, so privilege and credential behaviour needs much stricter proof than routine activity.
4. It builds several candidate operators (table, bitset, decision DAG, finite-state fragment, integer expression, PCB program, or a residual micro-model as last resort) and keeps the cheapest one that passes; Boundary Pressure and the Dual Oracle test it, and every failure becomes a stored counterexample.
5. The candidate runs in shadow beside DTL, then is promoted to a Knowledge Cell with a certificate level (A0 candidate up to A5 formally proven, only if actually proven).
6. At runtime the Boundary Index routes each Stage 1 event to matching cells in the Cell VM; events with no cell, a conflict or uncertainty go to DTL, and DTL audits a sample of cell decisions at a rate set by consequence and boundary distance.
7. Disagreement, drift, boundary violations and counterexamples raise a cell's stress

## RULES IT NEVER BREAKS

- A cell may never suppress mandatory evidence or turn a validated high-consequence result into a benign one to save resources; high-consequence suspicious behaviour is never normalized solely by repetition.
- Contradictory cells trigger abstention and a return to DTL, never an arbitrary winner; a lower-confidence cell cannot silently downgrade a higher-consequence result.
- PCB programs have no loops, recursion, dynamic allocation or unrestricted system calls; the Cell VM runs unprivileged with no arbitrary file or network access, and cell packages are signed, hashed and versioned.
- Incident evidence is immutable with respect to garbage collection and melting, and on corruption or ambiguity Stage 3 abstains and falls back to DTL or deterministic paths.

## WHERE IT RUNS

On the endpoint (the small 2 GB machine). The Knowledge Cell VM should run unprivileged where practical and sees only normalized Stage 1 data. Crystallization, adaptation and promotion run outside the privileged telemetry collector. The target is under about 25 MB of extra RAM normally, with an initial peak ceiling of 60 MB.

and lower its confidence; only the invalid part melts back to DTL (or the whole cell if its core rule fails), mixed cells split (fission), equivalent cells merge (fusion), and low-utility cells are garbage-collected.

#### IN THE EXAMPLE ATTACK

When the web server spawns the shell and the new process reads /etc/shadow, the Boundary Index looks for a validated cell; a cell such as NETWORK\_SERVICE -> CHILD\_INTERPRETER -> CREDENTIAL\_EXPOSURE could fire cheaply to raise risk, require evidence to be kept and request more observation, but if no cell covers this new chain, or it sits near a boundary, the event goes to Stage 2 DTL instead. Meanwhile the web server's ordinary request handling is answered by cheap cells, so DTL can focus on this incident, and Stage 3 will never crystallize this chain as 'normal' just because it repeats.

## 4 CBF + LUCID

Causal Belief Field and LUCID reasoning · Runs on the 2 GB machine

Stage 4 works out what is really happening in a suspicious incident. It keeps several competing explanations alive and only gives an answer once the evidence can actually tell them apart.

CBF = Causal Belief Field · LUCID = Latent Uncertainty and Causal Identification Dynamics  
· OBS = direct observation, DER = deterministic derivation, INF = model inference, CF = counterfactual result, EXT = external knowledge mapping, UNK = unknown/unobserved (the Epistemic Type System labels)

#### THE PROBLEM IT SOLVES

Ordinary security tools link alerts together and label the result malicious or not, even when the evidence is incomplete. On Linux the telemetry is always partial, so a tool can easily conclude "I did not see it, so it did not happen", or it can sound more certain than the evidence allows. Stage 4 replaces labelling with careful reasoning about which explanation could have produced the evidence. When the evidence cannot decide, it says so openly.

#### WHAT GOES IN

- Semantic events from Stage 1 (SSIR) and the current collection settings (which sensors are active and which events were dropped); from these Stage 4 builds its own visibility model, the Sensor Shadow
- Predictions of likely next behaviour from Stage 2 (DTL Future Cones)
- Trusted, compiled knowledge about known behaviour from Stage 3 (Knowledge Cells)

#### WHAT COMES OUT

- ← A resolution object for Stage 5: the surviving worlds, the possible

## THINK OF IT AS

*A careful detective keeps several suspects in mind. She asks what each suspect's story predicts should turn up next, checks the one clue that would rule a story out, and does not name a culprit until the other stories have been ruled out.*

consequences, the evidence trail, the uncertainty, whether the incident can be identified, and the gaps in the information

- ← Requests to Stage 1 to switch on one specific, cheap extra observation (for example, trace one process subtree)
- ← Feedback to Stage 3: stress signals that challenge a Knowledge Cell, and candidate new cells built from incident patterns it has resolved many times
- ← Unresolved or NON-IDENTIFIABLE cases passed to a human analyst, plus human-readable claims built from typed evidence

## MAIN PARTS

### Causal Belief Field (CBF)

A small, hard-capped set of competing "worlds". Each is a possible explanation of what the host is doing, and each lists the evidence it expects and the evidence it forbids.

### Sensor Shadow

A record of what PocketSec could not observe under its current settings. It lowers confidence and can ask Stage 1 to observe more.

### Discriminating Observation / Counterfactual Sensor Planning

Before paying for extra telemetry, it simulates whether that telemetry would actually tell the leading worlds apart. It only spends the budget if it would.

### LUCID engine

Updates the worlds as evidence arrives. It creates, kills, splits or merges worlds, challenges its own explanations, and decides when the evidence is strong enough to resolve the incident.

### Evidence Tension

Measures how badly each world fits reality: surprising events, expected evidence that is missing but should have been seen, and causes or timings that do not fit. A world can die from accumulated tension.

### Claim Compiler with Epistemic Type System

Builds human-readable claims only from evidence tagged as observed, derived, inferred, counterfactual, external or unknown. An optional tiny language model may only reword a claim that already exists, and its output is rejected if it adds anything unsupported.

## HOW IT WORKS

1. Collect the evidence from Stages 1-3 and build a small set of possible worlds. A new UNKNOWN world is created only if the existing worlds cannot explain a persistent, security-relevant leftover.
2. For each world, predict what evidence should appear next and what should never appear. Score each world's Evidence Tension. Missing evidence counts against a world only if the sensors should have seen it.
3. Run bounded "what if" tests: remove one event, rename a binary, add harmless context. These show which events really drive the conclusion (Responsibility Flux) and whether the conclusion is fragile.
4. If the worlds are still tied, pick the cheapest observation that would separate them and ask Stage 1 for it. Prune worlds that are clearly beaten by simpler ones.
5. Ask the identifiability question: can the affordable evidence tell the explanations apart? If yes, resolve. If not, report NON-IDENTIFIABLE, keep the case open, or escalate it before its Resolution Horizon (time, evidence and compute limit) runs out.
6. Compile the typed claims and send the resolution object to Stage 5. Send stress or crystallization signals back to Stage 3.

## RULES IT NEVER BREAKS

- Running out of memory or reasoning budget may prune low-consequence worlds, but it must never quietly turn uncertainty into "benign".
- Absence of evidence counts only when the visibility model says PocketSec should have observed it. Otherwise it stays UNKNOWN.
- Abstraction is allowed but factual amplification is not. For example, "read credential material" may become "possible credential access" but never "password stolen" without evidence for that stronger statement.
- Stage 4 is optional to core detection: if it crashes, Stages 1-3 keep detecting. Its acceptance test requires zero unsupported factual claims in authoritative output across the benchmark suite.

## WHERE IT RUNS

On the endpoint (the small 2 GB machine), with hard caps: normally 4-16 active worlds, a target of about 25-45 MB of extra memory, and an initial peak ceiling under 90 MB without the optional language model.

## IN THE EXAMPLE ATTACK

LUCID keeps several competing worlds alive for this incident. One: the web server was exploited, and an attacker is taking credentials and calling out. Two: legitimate automation that uses an unusual destination. Three: an UNKNOWN new mechanism. The /etc/shadow read is recorded only as [OBS] a read of credential material, which supports [INF] possible credential access. To separate the worlds cheaply, LUCID may ask Stage 1 for one targeted observation, such as the hash of /tmp/.x91 or a short trace of file access by that process and its children. It then hands Stage 5 the surviving worlds, with an explicit [UNK] for direct exfiltration.



---

## On the machine: responding

Decides whether to act, and makes sure any action is small, allowed and reversible.

### 5 SAFE + AEGIS + SENTINEL

Safe response under hard authority · Runs on the 2 GB machine

Stage 5 decides whether PocketSec should act on an incident, and if so, what the smallest safe, authorised, reversible and checkable action is. Doing nothing is always an allowed choice.

#### THE PROBLEM IT SOLVES

Automated response can do more harm than the attack. It can kill a critical service, destroy the evidence, or be tricked by an attacker or a poisoned log into harming its own machine. Stage 4 often cannot be sure which explanation is true, so acting as if it were certain is dangerous. Stage 5 exists mainly to enforce a safety boundary: being intelligent never gives permission, and every action must pass a strict, independent check.

#### THINK OF IT AS

*A hospital drug-dispensing system: a doctor (AEGIS) proposes a prescription, but a simple independent pharmacy check (SENTINEL) confirms the dose, patient identity and allergies before anything is dispensed. The system only offers approved drugs, in limited amounts, with a plan to stop them.*

#### WHAT GOES IN

- The Causal Belief Field resolution object from Stage 4: surviving worlds, consequences, uncertainty, identifiability and the evidence trail
- The Response Constitution and Mission & Asset Invariants: fixed safety laws and what must stay running or protected
- The typed Defensive Operator catalog and local policy, plus MITRE D3FEND used only as a vocabulary
- Local Effectiveness Memory: what has actually worked on this host before

#### WHAT COMES OUT

- ← One typed action (or NO ACTION) executed by the Transactional Executor under a narrow, expiring Capability Token
- ← A Human Decision Contract proposal when the action needs human approval: its effect in each world, benefit, collateral damage, rollback and a safer alternative
- ← An immutable ResponseRecord and a preserved pre-action evidence bundle
- ← The Intervention Residual (how the outcome differed from the prediction), fed back to Stage 4 CBF, Stage 3 CRYSTAL and effectiveness memory. Outcomes also flow on to Stage 6 learning

**Response Constitution**

Fixed laws that cannot be changed at runtime. They cover safety, authority and prohibited actions (for example, no offensive or counterattack behaviour).

**AEGIS planner**

The unprivileged planner. It scores each candidate across all of Stage 4's plausible worlds using a narrow Counterfactual Response Twin and Intervention Cones, then removes dominated options (Pareto / regret) to propose the least harmful sufficient action.

**Defensive Operator Algebra**

The only actions that exist: typed operators from O0 Observe and O1 Preserve up to O7 Irreversible, which is never autonomous. There is no arbitrary shell command, so injected text cannot become a privileged command.

**SAFE Action Field**

A small, bounded list of candidate interventions. Each lists its target, preconditions, expected security and operational effect, reversibility, rollback plan, authority class and lease.

**SENTINEL Constraint Kernel**

A tiny, independent verifier that sits between the reasoning and the privileged actions. It checks schema, authority, scope, target, preconditions, protected assets, evidence preservation, rollback and expiry, then says PASS or DENY. AEGIS cannot bypass it.

**Transactional Executor**

A minimal privileged component with no machine learning, no language model and no network API. It prepares, commits, verifies and rolls back one action, and re-checks the target's identity just before acting.

## HOW IT WORKS

1. Take Stage 4's worlds without collapsing their uncertainty, and generate a few typed candidate actions, including observe, preserve, defer and do nothing.
2. For each candidate, predict its effects in every plausible world: security benefit, service damage, evidence loss, attacker adaptation, and unknown side effects (Action Shadow). An action is allowed only if every world where it would cause unacceptable harm has been ruled out or explicitly accepted by policy (Response Identifiability).
3. AEGIS drops dominated actions and chooses the minimum effective intervention from the options that remain.

## RULES IT NEVER BREAKS

- No model confidence grants execution authority, and no natural-language text can become a privileged command (no arbitrary shell).
- SENTINEL can independently deny any action and cannot be bypassed. UNKNOWN is never treated as permission for destructive action, and NO ACTION must always be possible.
- Every autonomous action must be scoped, time-limited, verifiable and reversible. An action that cannot be rolled back must cross a higher authority boundary, and required evidence is preserved before acting where feasible.

Bigger or harder-to-undo actions need higher authority, from A0 (read-only) up to A5 (administrator-only). AX marks actions that are never allowed to run autonomously. Many actions need human approval through the Human Decision Contract.

4. SENTINEL verifies the proposed action. If it passes, a narrow, expiring Capability Token is issued, and the Evidence Preservation Gate saves the evidence the action might destroy.
5. The executor prepares (re-checks identity and preconditions, captures rollback state), commits one operator, verifies that the intended effect really happened, and rolls back if it failed or caused harm. Temporary restrictions run on leases that expire unless evidence justifies renewal.
6. Measure the Intervention Residual, plan a step-by-step safe recovery, and feed the results back to Stages 4 and 3 and to effectiveness memory. Repeated verified responses may become Response Knowledge Cells, which melt again when conditions drift.

- If Stage 5 fails, Stages 1-4 must keep monitoring and detecting. On budget exhaustion Stage 5 prunes or escalates; it never relaxes safety.

#### WHERE IT RUNS

On the endpoint (the small 2 GB machine). The planner runs unprivileged and a tiny SENTINEL plus executor runs privileged. The target is under about 35-50 MB of extra memory normally and an initial peak ceiling under 110 MB. High-impact responses are learned only offline, in a lab, or from human-approved actions, never by trial and error in production.

#### IN THE EXAMPLE ATTACK

Given Stage 4's worlds, AEGIS first chooses low-risk steps. It preserves evidence (a snapshot of the incident metadata and the state of the /tmp/.x91 process). It then proposes a temporary, leased restriction on that one process's connection to 10.0.0.9, or suspending the process with a validated resume. It does not contain the web server itself, because service containment (O5) needs a human by default, and if the web server is listed as a critical service in the mission invariants it may not be interrupted autonomously at all. SENTINEL checks the action against scope, authority and the mission invariants. Before acting, the executor re-checks the target's identity (PID, start time, executable hash). Afterwards it verifies that the connection was actually blocked, and if it was not, it rolls back or escalates to a human.

---

# On the machine: learning over time

The only way anything changes what the machine trusts.

## 6 HELIOS + MNEMOSYNE

Quarantined long-term learning and memory · Runs on the 2 GB machine

Lets PocketSec keep learning from its own experience for months or years, while making sure attackers, normal change and memory limits cannot corrupt or erase what it already knows.

APF = Adaptive Plasticity Field · KCG = Knowledge Conservation Gate

### THE PROBLEM IT SOLVES

A security system that learns can be taught the wrong thing. An attacker can repeat a harmful action until it looks normal, new learning can wipe out old skills (catastrophic forgetting), and learned memory can grow without limit on a 2 GB machine. Stage 6 exists to treat learning itself as a privileged, security-sensitive operation: every change must earn its place with evidence and must be reversible.

### THINK OF IT AS

*Like a hospital updating its treatment guidelines: a new observation goes to a review committee, is checked against past cases and tried in a small pilot before it changes practice, and the old guideline is archived so it can be restored.*

### WHAT GOES IN

- Stage 1 telemetry quality and sensor context, and Stage 2 dynamics / Future Cones
- Stage 3 Knowledge Cell outcomes and Stage 4 resolved or unresolved security worlds
- Stage 5 actions and their verified outcomes, plus explicit analyst labels if available
- Foreign knowledge from Stage 7, which always arrives as a candidate that enters Stage 6 quarantine
- Candidates from the offline research stages: DiscoveryPackages from Stage 8 and SuccessorPackages from Stage 9, which enter the same quarantine as any other experience

### WHAT COMES OUT

- ← Promoted, trusted knowledge (baselines, thresholds, Knowledge Cells, small models) handed back to the live Stage 1-5 pipeline
- ← Knowledge Fossils and Knowledge Lineage DAG records, so every learned item has a history and a known-good version to return to

- ← Learning rollback that restores a known-good state after a bad update, while keeping the record of why it failed
- ← Validated knowledge that Stage 7 may share with other hosts, but only when explicitly configured (no fleet export by default)

## MAIN PARTS

### Experience Gateway + Experience Quarantine

Packs each outcome from Stages 1-5 into an ExperienceCapsule and holds it in quarantine, because no raw experience may directly change a trusted model, baseline or Knowledge Cell.

### Shadow Mind

Runs a candidate model or knowledge state on the same inputs as the real system, but with zero authority, so its predictions can be compared without affecting any decision.

### Semantic Homeostasis

Protects core security meanings (privilege boundaries, credential material, executable trust, persistence, evidence integrity, response authority) so that frequency alone can never make them look normal.

### HELIOS Evolution Chamber

An isolated workspace where candidate changes are built, from cheap threshold tweaks up to new Knowledge Cells or small classifier heads, without touching the live system.

### Knowledge Conservation Gate (KCG)

A nine-check barrier (integrity, historical replay, new utility, counterfactual replay, adversarial, calibration, resource, shadow, rollback) that a candidate must pass before promotion; failing any one check sends it back to quarantine.

### MNEMOSYNE Consolidator

Does memory maintenance rather than training: removes duplicate episodes, merges or splits Knowledge Cells, ages weak knowledge, fossilizes valuable knowledge, retires obsolete knowledge and brings back dormant knowledge when an old context returns.

## HOW IT WORKS

1. The Experience Gateway turns each resolved outcome into an ExperienceCapsule that records its evidence, source (provenance), epoch, privacy class and any contamination flags.
2. Quarantine removes duplicates, scores where the evidence came from, checks

## RULES IT NEVER BREAKS

- Raw telemetry can never directly modify trusted cognition; all experience passes quarantine first.
- Repetition is not truth: protected security meanings do not adapt merely because something happens often.

whether repeated evidence is really independent (one compromised process repeating itself counts as one source), estimates poisoning suspicion, and sorts it into trusted-candidate, uncertain, hostile-suspect or discard. Only high-value episodes are kept in bounded memory.

3. HELIOS builds a candidate change in isolation, cheapest kind first. The Adaptive Plasticity Field decides which small part may change and by how much; everything else stays frozen.
4. The candidate competes against old validated knowledge on the same replay set, is rehearsed on altered copies of past cases (renamed actors, shifted timing, dropped telemetry, decoy events), runs in the Shadow Mind, and must pass all nine checks of the Knowledge Conservation Gate.
5. It then moves through a promotion ladder: QUARANTINED, CANDIDATE, OFFLINE-VALIDATED, SHADOW, CANARY, TRUSTED. At canary it has only limited influence as secondary evidence; a bad canary automatically reverts, and rollback restores a known-good Knowledge Fossil.
6. Over time MNEMOSYNE ages knowledge according to its Epistemic Half-Life (stable, recurring knowledge lasts longer), opens a new epoch on software or configuration changes instead of overwriting old knowledge, and runs only when memory, CPU and disk allow, so it never blocks detection.

- Every promoted knowledge object needs complete provenance and lineage, and a known-good fossil must exist for rollback; unknown lineage blocks promotion.
- Shadow or canary candidates can never gain Stage 5 response authority, and a crash in learning leaves trusted cognition unchanged.

#### WHERE IT RUNS

On the 2 GB endpoint for cheap work (quarantine, trust scoring, episode selection, small baseline updates, sampled Shadow Mind, fossil rollback); classifier retraining, distillation, large teacher models, architecture search and full historical replay run offline on a bigger development machine.

#### IN THE EXAMPLE ATTACK

Stage 6 does not act during the incident. Afterwards the resolved chain (web server spawns a shell, hidden file /tmp/.x91 is written and run, /etc/shadow is read, connection to 10.0.0.9) and Stage 5's verified outcome become an ExperienceCapsule in quarantine; because reading password hashes touches the protected credential-material meaning, Semantic Homeostasis ensures that even if an attacker repeats this chain many times it can never be learned as normal. HELIOS may propose a candidate Knowledge Cell for the chain, which is rehearsed with renamed files and shifted timing, run in the Shadow Mind and passed through the Conservation Gate and a canary before it becomes trusted, with a fossil kept for rollback.

Note on the word 'shadow': Stage 6's Shadow Mind (a candidate running with zero authority) is related to Stage 3's shadow execution (a new cell running silently beside DTL). Stage 4's Sensor Shadow (what PocketSec could not see) and Stage 5's Action Shadow (the unknown side effects of an action) are different ideas that share the word.

# Beyond one machine

Machines share knowledge, but each one keeps the final say.

## 7 ORPHEUS + HIVELOCK

Zero-trust knowledge sharing between machines · Runs across many machines

Lets many PocketSec machines share small, privacy-protected pieces of security knowledge, while each machine keeps the final say and treats everything it receives as untrusted until it checks it itself.

ECHO = Evidence-Causal Heterogeneity-aware Orchestration

### THE PROBLEM IT SOLVES

One machine sees only part of an attack; a group of machines can spot campaigns that no single host could. But ordinary federated learning has known weaknesses: machines differ from each other, shared model updates can leak private data, and fake or compromised peers can poison what the group believes. Stage 7 must gain the group's knowledge without ever letting a crowd, however large or well signed, override a host's own defences.

### THINK OF IT AS

*Like hospitals in different cities sharing anonymised case reports: each hospital reads its peers' reports but re-checks them against its own patients before changing practice, and a hundred copies of one report do not count as a hundred independent confirmations.*

### WHAT GOES IN

- Validated local knowledge and resolved incidents from Stage 6, to be distilled into shareable capsules
- Signed Knowledge Capsules from peer hosts (antibodies, novelty, campaign fragments, negative evidence, drift notices, revocations, optional model deltas)
- The local host's own context (role, software epoch, sensor visibility, policy) used to judge how relevant foreign knowledge is
- Compact discoveries from Stage 8, shared as Knowledge Antibodies that other hosts still treat as untrusted

### WHAT COMES OUT

- ← Privacy-distilled, signed Knowledge Capsules sent to peers
- ← Foreign knowledge handed to the Stage 6 Experience Quarantine as a candidate only, never directly into trusted cognition
- ← Candidate distributed-campaign hypotheses that survive falsification, attached as candidate evidence
- ← Verified revocations that mark descendant knowledge suspect and trigger local Stage 6 re-evaluation and targeted rollback

### Knowledge Capsule Compiler + Privacy Distiller

Turns local knowledge into a compact, signed claim with its evidence history and uncertainty, after replacing usernames, paths and other host details with general classes and testing for privacy leakage.

### HIVELOCK Ingress

The single entry gate for foreign knowledge: it checks size and rate, format, signature, replay and expiry, source, privacy policy, Sybil and dependence risk, poisoning suspicion and relevance, then sends the object into Stage 6 quarantine.

### ECHO

The collective reasoning algorithm: it weighs each capsule by host similarity, evidence independence and poisoning suspicion, and updates group belief only from independent evidence that survived local checks, never by majority vote.

### Antibody Forge

Strips host identity from a resolved incident and keeps only the smallest behaviour pattern that still reliably marks it as malicious, so it transfers to other machines.

### Dependence/Sybil Graph

Maps which contributions share a source (same software image, same upstream detector, same administrator, common parent capsule) so that copies of one source are counted roughly as one piece of evidence.

### Consensus Falsifier

Challenges every strong group belief with rival explanations (coincidence, a shared benign software update, admin automation, a telemetry glitch, poisoned or colluding peers) and looks for the cheapest privacy-safe observation that tells them apart.

#### HOW IT WORKS

1. Export: a resolved incident is reduced to a minimal invariant by the Antibody Forge, host-specific detail is stripped by the Privacy Distiller, leakage is tested, and the result is signed as a Knowledge Capsule. The Privacy Budget Plane tracks how much has been disclosed over repeated exchanges.
2. Import: every incoming object passes through the HIVELOCK Ingress checks and then enters the receiving host's own Stage 6 quarantine, Shadow Mind, Knowledge

#### RULES IT NEVER BREAKS

- Stage 7 can never directly change trusted Stage 1-5 cognition or trigger a Stage 5 action; all foreign knowledge goes through Stage 6 quarantine, even when signed.
- Majority vote is never ground truth, and one source cannot gain more influence by creating many identities.
- Raw command history, credentials, sensitive file names and raw host telemetry are not shared by default.
- Losing the Stage 7 network must not reduce local protection; every imported

Conservation Gate and local canary. No bypass path exists.

object must be locally rejectable, expirable and revocable.

3. ECHO scores each capsule: Epistemic Distance (how similar the sending host's role, software and sensors are), dependence mass (duplicates discounted) and poisoning suspicion combine into Knowledge Gravity, which decides whether the capsule deserves expensive local validation.
4. Periodically, the Partial-World Reconstructor joins weak fragments from different hosts into a candidate distributed attack, recorded in a Campaign Hypergraph, and the Collective Novelty Engine looks for patterns that are rare across similar hosts.
5. The Consensus Falsifier tests each campaign hypothesis against benign and adversarial counter-explanations before it counts as evidence.
6. The Cross-Host Lineage DAG records where every shared object came from; if one is later found bad, the Revocation Plane verifies the revocation, marks its descendants suspect, and each host re-checks and rolls back only the affected

#### WHERE IT RUNS

Across a fleet of machines, and only as an optional add-on: each 2 GB endpoint runs its own HIVELOCK ingress and stays fully functional offline, while federated baselines and large peer simulations (up to 10,000 simulated peers) run offline on research machines.

#### IN THE EXAMPLE ATTACK

Stage 7 does not see this incident live. Once resolved, the Antibody Forge can turn it into a signed ANTIBODY capsule in which exact details such as /tmp/.x91 and 10.0.0.9 are replaced by general classes (a hidden temporary executable, an outbound destination) and no password material is ever exported; other hosts treat it only as a candidate that must pass HIVELOCK and their own Stage 6 checks. If other hosts report weak fragments of the same pattern, such as destination novelty, the Partial-World Reconstructor may propose a cross-host campaign, which the Consensus Falsifier tests against coincidence or a shared benign software update before it counts.

# 12 CONSTELLATION Ω

Sovereign collective security science · Runs across many machines

Lets many separate PocketSec machines share what they have learned about attacks, like a community of skeptical scientists, while each machine still decides for itself what to trust and do.

SKA = Security Knowledge Atom · PCK = Proof-Carrying Knowledge · FCK = Falsification-Carrying Knowledge · NKA = Negative Knowledge Atom

## THE PROBLEM IT SOLVES

One small machine sees only its own attacks, so it learns slowly while attackers adapt quickly. Ordinary sharing (threat feeds, or federated learning, where a central model is trained from everyone's updates) either leaks private data or lets one bad or hacked source push harmful changes to every machine. Stage 7 already shares knowledge between machines. Stage 12 is a later and stricter design for the same problem, and it must beat Stage 7 as a baseline. Its aim is to make the group learn faster than attackers without any peer, coordinator or majority vote ever gaining control over a machine: 'Global knowledge -> local verification -> local decision', never 'global command -> endpoint execution'. (Stage 12's ORACLE and GENESIS are not the Stage 8 and Stage 9 engines of the same name.)

## THINK OF IT AS

*It works like scientists publishing papers: each lab shares its findings with the method and the ways it could be proven wrong, and other labs repeat the experiment themselves before they believe it, instead of doing whatever the loudest or most popular lab says.*

## WHAT GOES IN

- Knowledge Atoms from other PocketSec nodes: short, abstract claims about an attack mechanism, bundled with proof material, counterexamples and a statement of how the claim could be disproved
- This node's own locally validated discoveries from Stages 1-11, which may be abstracted and shared
- Optional platform evidence (attestation) about the state of a contributing machine
- The organization's own rules: what may be imported or exported, privacy classes, accepted trust roots

## WHAT COMES OUT

- ← Foreign knowledge that passed local checks goes through SANCTUM to GENESIS, is recompiled locally via FORGE, then checked by Stage 10 AEGIS PRIME and Stage 11 ETERNA before becoming part of the node's own 'local phenotype' (its running detectors)
- ← Privacy-reduced Knowledge Atoms, counterexamples and signed revocations sent out to peers
- ← Unconfirmed high-severity HYPOTHESIS atoms, which may only raise how carefully a node watches its existing telemetry, never block or remediate
- ← Isolation modes from ECLIPSE (restricted import up to full isolation) when the

## MAIN PARTS

### VEIL

Turns a raw local incident into a privacy-safe mechanism description by stripping identifiers and blurring precise time and location, then tests it for leakage, so what is shared is 'how the attack works', not a log.

### IMMUNE

Defends against poisoned or colluding contributions using plausibility checks, lineage analysis, contradiction search, rate limits, quarantine, staged activation and revocation.

### GENESIS

Rebuilds accepted knowledge from its meaning into the node's own code, so the sender's executable is never trusted or reused; the same idea may compile differently on a 2 GB ARM box and a 64 GB server.

### QUORUM (with TRUSTGRAPH)

Weighs how much independent support a claim really has; copies, shared ancestry and cloned identities count for little, and trust is tracked per peer, per topic and per time period instead of as one global reputation score.

### SANCTUM

A sandbox where foreign knowledge is parsed under a strict, small format, checked for signatures, privacy and types, replayed against local history and tested before it can even become a shadow candidate.

### ECLIPSE

A sovereignty circuit breaker that can cut any or all collective connections at any time, while the node's own Stage 1-11 protection keeps working.

## HOW IT WORKS

1. A node that has learned something locally uses VEIL (and SHARD, which splits knowledge into small fetchable pieces) to turn it into a Knowledge Atom: an abstract claim plus proofs, counterexamples and falsifiers, with no raw logs.
2. DIPLOMAT carries atoms over any transport (USB, LAN, relays, optional Internet); on the receiving side ORACLE, ENTROPY and MARKET decide which incoming claims are worth the scarce validation effort, favouring severe, novel, locally relevant and under-replicated ones and dropping duplicates.

## RULES IT NEVER BREAKS

- No received knowledge may directly execute or change response policy: there is no RECEIVED -> ACTIVE shortcut, no remote execution authority and no remote policy override (target Sovereignty Loss = 0).
- No trust by popularity: many identities or many copies of one source are not many independent observations, and safety must not depend on more than half of nodes being honest.
- No raw telemetry export by default: the shared object is an abstract mechanism, and privacy leakage must be measured, not just claimed.

3. TRUSTGRAPH, QUORUM, ATTESTOR and IMMUNE judge the evidence: how independent the supporters really are, whether the lineage looks poisoned or colluding, and whether contradictions exist; node count alone never counts as proof.
4. RESONANCE, CAUSALITY and MOSAIC compare and combine claims: they check whether independently found mechanisms match, whether a mechanism still holds on a different kind of host, and whether separately validated pieces form a larger attack chain (each combination needs new tests); DIVERGENCE keeps the population from all running the same detector.
5. SANCTUM tests the surviving knowledge locally; GENESIS recompiles it into local code, which then passes Stage 10 AEGIS PRIME and Stage 11 ETERNA. It moves through fixed states (DISCOVERED -> QUARANTINED -> REPRODUCED -> CONTESTED/REFINED -> LOCALLY\_VALIDATED -> SHADOW -> ACTIVE -> FOSSIL) and can never jump straight from received to active.
6. If knowledge later proves poisoned, the PLAGUE process finds its descendants through lineage, quarantines and revalidates them, rolls back local detectors if needed and shares a signed counterexample; ECLIPSE can isolate the node at any time.

- No collective dependency for local protection: if the network is absent or isolated by ECLIPSE, Stage 1-11 protection continues, and the endpoint's Stage-12 state stays small and bounded for the 2 GB target.

#### WHERE IT RUNS

Across a fleet of machines. Each 2 GB endpoint runs only a small part (a tiny DIPLOMAT client, an atom/provenance checker, a bounded trust cache and a SANCTUM job interface, with most atom storage on disk); heavy replication and the six research laboratories (SIBYL, PLAGUE, PRISM, ECOSYSTEM, MENDEL, GALILEO, simulating up to 100,000 nodes) run offline on bigger machines, and no cloud control plane is required.

#### IN THE EXAMPLE ATTACK

Stage 12 does not stop this incident live; the local Stage 1-11 pipeline does. Afterwards, VEIL could turn it into an abstract atom such as 'web-server process -> shell -> hidden file written in /tmp and run -> password-hash file read -> outbound connection', stripping identifiers like the exact file name, the address 10.0.0.9 and precise timestamps, and attaching tests and counterexamples. Other nodes would quarantine that atom, reproduce it in SANCTUM, recompile it with GENESIS and adopt it only after their own checks; they would never block anything simply because a peer said so.



---

## Offline research

Runs on a bigger computer. It can only send candidates, which still go through Stage 6.

### 8

## PROMETHEUS + ORACLE + FORGE

Offline discovery lab · Runs offline

Stage 8 is PocketSec's offline research lab. It takes behaviour the system cannot yet explain, proposes rival theories about it, tries hard to prove each theory wrong, and turns any theory that survives into a very small detector for the 2 GB machine.

AION = Architecture Invention and Optimization Nexus · MIR = Mechanism Intermediate Representation · EIG = Expected Information Gain · DCR = Discovery Compression Ratio

#### THE PROBLEM IT SOLVES

Stages 1 to 7 detect, reason, respond, learn and share knowledge, but some observations still fit none of their explanations. The doc calls these leftovers residuals. Stage 8 needs to turn residuals into new defensive knowledge without trusting an AI's story just because it sounds good, and without becoming a system that attacks real machines. The doc is clear that this is defensive research only: no theory it produces gives permission to act on any system.

#### THINK OF IT AS

*It works like a medical research lab studying cases that doctors could not diagnose: it lists possible causes, runs safe tests to rule each one out, and publishes the surviving finding as a cheap, simple test that clinics can use, and the test still goes through trials (Stage 6) before any clinic adopts it.*

#### WHAT GOES IN

- Residuals: patterns in observations that the knowledge of Stages 2, 3, 4, 6 and 7 cannot explain, plus Stage 5 responses whose results differed from prediction (for example sensor surprises, unexplained timing, a response that did not work as expected, or telemetry gaps)
- The PocketSec replay corpus of stored episodes
- Distributed residuals and independent counterexamples from Stage 7 (the collective of machines)

#### WHAT COMES OUT

- ← A signed DiscoveryPackage (the mechanism, its detector, its full history, the conditions where it fails, and its resource cost), sent to Stage 6 Quarantine as a candidate only. It then passes through Shadow Mind, the Conservation Gate and a Canary before it can be trusted
- ← Compact discoveries that Stage 7 can share with other hosts as Knowledge Antibodies, which those hosts still treat as untrusted

- ← Scientific memory kept for later: open questions, falsified ideas (negative results), counterexamples and 'discovery fossils' (mechanisms that were validated)

## MAIN PARTS

### Residual Observatory + Discovery Priority Field

Measures what current knowledge cannot explain, then ranks which leftovers are worth research by weighing security impact and recurrence against cost and safety risk.

### ORACLE

Picks the cheapest safe experiment that best separates the competing theories. Examples are replaying stored history, making small edits to past evidence (renaming files, changing timing, deleting sensor data), or running an authorized isolated lab.

### FORGE

Compiles each surviving theory into several cheap forms (a rule, a finite-state machine, a Knowledge Cell, a tiny model). A 'Representation Tournament' keeps the smallest one that still meets the security bar. The most sophisticated form does not win automatically.

### PROMETHEUS

Generates several competing theories, each written in a strict, limited 'Mechanism Grammar'. Several generators contribute, including a symbolic enumerator, an analogy engine and a 'null generator' that proposes innocent explanations. An LLM is only one generator and gets no special weight.

### Benign Doppelgänger Engine

For every theory that looks malicious, it builds the strongest innocent look-alike, such as a backup, a software update or an admin script. A theory only counts as strong if it can tell the attack apart from this innocent twin.

### AION

An extension that looks for smaller ways to represent security-relevant information, not only for new rules. For example, it tries to reduce many surface-different event sequences to one canonical 'motif'. It can propose these designs but can never deploy them itself.

## HOW IT WORKS

1. Find the residual: compare what was observed with the best explanation the earlier stages can offer, then sort the gaps into types (observation, causal, timing, world, response, learning, collective, visibility).

## RULES IT NEVER BREAKS

- Defensive research only: no hypothesis gives permission to act on an external system, and Stage 8 creates no direct path to Stage 5 response authority.
- An LLM may propose ideas but cannot define ground truth. Free-form LLM text is never the official scientific record.

2. Rank the gaps and spend research effort only on those that matter, so the lab does not chase every novelty.
  3. PROMETHEUS writes several rival hypotheses as typed 'Hypothesis Genomes'. Each genome must list the observations that would prove it false. The system deliberately keeps incompatible ideas alive early on.
  4. ORACLE runs safe experiments to eliminate theories: replay, counterfactual edits, 'metamorphic' checks (the answer should not change when only names change) and adversarial robustness tests. A Theory Ledger records every result, and the Causal Identifiability Gate may conclude that the true cause cannot be determined.
  5. Survivors must repeat across different hosts, time periods and campaigns. A novelty audit checks whether the finding is really new or only a rediscovery of known ATT&CK/Sigma/YARA knowledge.
  6. FORGE compiles the winning theory into a kilobyte-scale detector (the endpoint budget allows up to a few MB for added detectors) and packages it as a signed DiscoveryPackage, which goes to Stage 6 quarantine on the endpoint
- Every accepted theory must state what would falsify it. UNKNOWN / UNIDENTIFIABLE is an acceptable scientific result, and the system must not invent certainty.
  - Every discovery is only a candidate until it passes Stage 6 (quarantine, shadow, conservation testing, canary). Active attack emulation happens only in an isolated, authorized sandbox, and production systems are limited to observation and replay by default.

#### WHERE IT RUNS

Mostly offline, on a larger development/research machine. The endpoint keeps only small pieces: a residual tracker, a discovery-fossil index and the compiled detectors (target below 20-35 MB extra). Full PROMETHEUS/ORACLE research is off the endpoint by default.

#### IN THE EXAMPLE ATTACK

Stage 8 would not act on this incident live. If incidents like it (web server spawns a shell, a hidden /tmp file is run, /etc/shadow is read, an outbound connection follows) keep leaving residuals, PROMETHEUS would propose a typed mechanism such as execution → file write → execution → sensitive read → new destination, and set it against innocent 'doppelgängers' like an admin or backup script. ORACLE would then test it on replayed history, for example by renaming /tmp/.x91 or changing the timing, to confirm the theory depends on behaviour and not on names. FORGE would compile the survivor into a tiny finite-state-machine detector and send it to Stage 6 quarantine as a candidate. The novelty audit would probably label it known ATT&CK behaviour rather than a new discovery.

# 9 ONTOGENESIS

Search for the minimum sufficient computation · Runs offline

Stage 9 is an offline search for the smallest and cheapest computing structure that can still make the security decisions that matter on a 2 GB machine. That structure covers what to remember, what to forget, which operations to use and how the parts fit together.

MSSC = Minimum Sufficient Security Computation · MSDL = Minimum Security Description Length · QD = Quality-Diversity · MIR = Mechanism Intermediate Representation

## THE PROBLEM IT SOLVES

Earlier stages use architectures that people designed by hand, and those designs may carry more state, memory and computation than security actually needs. Stage 9 asks whether the needed computational structure can itself be discovered by experiment instead of assumed. The doc calls this target Minimum Sufficient Security Computation. The answer may be a neural model, but it may equally be a state machine, a sparse memory, a statistical process or a hybrid. All of this must happen without any research engine being able to rewrite the production system directly.

## THINK OF IT AS

*It works like plant breeding under strict inspection: many small variants are bred and trimmed for different climates (server, desktop, low-memory machine), a hostile inspector tries to break each one, and only varieties that pass inspection and quarantine are allowed into the real field.*

## MAIN PARTS

### LAPLACE

Discovers the laws a detector follows: what small state it keeps, how that state

## WHAT GOES IN

- Datasets and replay corpora of security episodes
- Unexplained residuals from Stage 8
- Collective evidence from Stage 7
- Real resource measurements from a reference 2 GB Linux machine (memory, CPU cycles per event, latency, event loss)

## WHAT COMES OUT

- ← A signed, rollbackable SuccessorPackage (the computational genome, its lineage, compiled artifacts, resource contracts, security metrics, counterexample tests and known failure domains). It is compiled through MIR/FORGE and must pass Stage 6 quarantine (and later Stage 10 assurance)
- ← Host-specific 'phenotypes': different verified configurations for web servers, desktops, databases, containers or very-low-memory machines
- ← Promoted 'computational laws' and primitives, plus fossils of failed designs, which can seed new Stage 8 hypotheses

### CHRONOS

Discovers memory and forgetting rules. It decides what to keep, for how long, and at which timescale (seconds, minutes, hours, days), and it forgets an

changes with each event, how it reads out a security answer and how it adapts.

## DAEDALUS

Puts the discovered state, operators, memory and learning rules together into a complete candidate system, type-checks it against resource limits, compiles it and tests it.

## GAIA

Keeps a varied population of good designs, one best design per niche (host role, RAM size, available sensors, latency need), instead of searching for one universal winner. It uses the established MAP-Elites method as its baseline.

item only after a test shows the loss would not change future security decisions.

## GENESIS

Creates variations of candidate systems by replacing, merging, splitting, specializing or deleting parts. Every generation includes deletions, so parts that add nothing are removed ('subtractive evolution').

## ARGUS

The adversary. It tries to break every candidate through poisoned data, evasion, corrupted state, lost sensors, resource starvation, tampered artifacts, and 'fitness hacking', where a design learns the benchmark instead of the security task.

### HOW IT WORKS

1. ONTOGENESIS, the meta-controller, describes each candidate as a typed 'Computational Genome' (state space, operators, memory, forgetting, learning, uncertainty, routing and resource laws) rather than as arbitrary source code.
2. LAPLACE, CHRONOS and the Primitive Foundry propose state and transition laws, memory and forgetting rules, and a small alphabet of basic operations (COUNT, DECAY, RARE, FIRST\_SEEN, and others). The Security Renormalizer and Symmetry Laboratory then find which details can safely be abstracted or ignored, such as user names, PIDs and file paths.
3. DAEDALUS assembles these into executable systems. GENESIS varies them and removes parts that contribute nothing. GAIA keeps the best system for each type of host.

### RULES IT NEVER BREAKS

- No research engine can directly replace the trusted production architecture, and no generated computation can create a new Stage 5 response authority path.
- Every security conclusion must keep an evidential path. UNKNOWN and UNIDENTIFIABLE remain valid outcomes.
- Resource contracts are hard limits, not preferences. Constitutional safety rules cannot be evolved by GENESIS, and no generated code is synthesized on the endpoint (self-repair only swaps in components that were validated beforehand).
- Every successor must be rollbackable and must pass independent compilation, benchmarking and quarantine. Physics-style ideas (symmetry, conservation, phase change) are kept only if they beat simpler baselines.

4. ARGUS attacks every candidate, and each one is run on a real 2 GB reference machine so its memory and CPU cost are measured, not estimated.
5. The Law Promotion Gate accepts a recurring structure only if it is rediscovered independently, reproduces across hosts and time periods, beats simpler baselines and has a documented failure domain.
6. Winners are compiled through MIR/FORGE into a signed SuccessorPackage and admitted only through Stage 6 quarantine. On the endpoint, a Homeostatic Runtime keeps memory and CPU within bounds by switching between regimes such as survival, reflex and adaptive. When it switches to a lower regime it reports which

#### WHERE IT RUNS

Offline by default on a research machine: the Stage 9 research engines use 0 MB on the endpoint. Only the result runs on the 2 GB endpoint: a small discovered state, a motif/state-machine 'ecology' and the Homeostatic Runtime controller, with memory targets such as under 5 MB for the controller.

#### IN THE EXAMPLE ATTACK

Stage 9 does not handle this incident live. It uses replays of incidents like it to find the smallest computation that still catches the whole chain. For example, the Symmetry Laboratory would learn that the exact name /tmp/.x91 and the PIDs do not matter, while the privilege-sensitive read of /etc/shadow followed by a new outbound destination does. CHRONOS would decide how long the 'web server spawned a shell' fact must be remembered so that the later read and connection can be linked to it. The winning design, for instance a small state machine for the web-server phenotype, would go to Stage 6 as a SuccessorPackage only after ARGUS fails to break it.

# Keeping it trustworthy

Watches the whole system now, and keeps it working over years.

## 10 AEGIS PRIME

Engines: AEGIS, THEMIS, JANUS, MIRROR, SENTINEL, PARADOX, PHOENIX · Runs on the 2 GB machine

Stage 10 checks, all the time, whether PocketSec's detectors still deserve trust. It makes every alert show the evidence behind it, and it moves the system to a safer, simpler mode when that trust runs out.

ECE = Expected Calibration Error

### THE PROBLEM IT SOLVES

Stage 9 lets PocketSec discover its own detection programs. A program found by a machine should not be trusted just because it once scored well in a test. Its evidence can be weak, it can run in conditions it was never tested in, and it can quietly change when it is compressed or updated. Stage 10 exists to catch these problems, and to do it without becoming a second heavy security product on a 2 GB machine.

### THINK OF IT AS

*It is like a hospital's quality-assurance office: every diagnosis must cite its lab results, a doctor from a different specialty re-checks important cases, and when equipment fails the ward switches to basic, well-tested procedures instead of guessing.*

### WHAT GOES IN

- Trusted detector artifacts promoted from Stage 9 and compiled by FORGE, plus their contracts
- Live detector conclusions and the chain of sensor evidence behind them
- Runtime health: memory, CPU, queues, storage, sensor coverage and the current environment (kernel, hardware, workload, time)
- Delayed ground-truth labels that arrive after an investigation

### WHAT COMES OUT

- ← Proof-carrying alerts: each alert carries its evidence certificate, provenance, assurance vector and uncertainty, and goes to the response and analyst side
- ← Assurance-degraded signals and safe-state changes (PHOENIX steps down from P3 Full to P2, P1 or P0), each stating the coverage lost
- ← Assurance acceptance or rejection for updates and promotions, used by the Stage 6 gates and required by Stage 11 before a successor is legitimate
- ← A Failure Atlas, an immutable prediction ledger and a transparency log for audit

## MAIN PARTS

**AEGIS**

The central assurance controller. It combines all the checks into one assurance state, made of several separate scores (functional, safety, resource, evidence, calibration and others), and never collapses them into a single hidden number.

**SENTINEL**

Small runtime monitors on the endpoint. They check resource limits, sensor coverage, that every alert has evidence, that no forbidden authority path is used, and that the machine is still inside the conditions where the detector was validated.

**MIRROR**

Checks that a compressed or recompiled detector still means the same thing as the original: the same decision, severity, evidence and uncertainty, not just the same average accuracy.

**THEMIS**

Writes executable contracts (what a detector needs and what it promises) and applies the epistemic type system, which labels every piece of evidence DIRECT, DERIVED, INFERRED, CORROBORATED, NEGATIVE, MISSING or CONTRADICTED.

**JANUS**

An independent shadow detector built from a different kind of computation, for example a tree ensemble shadowing a state machine. On selected high-value episodes it gives a second opinion. Its agreement counts only as much as it is truly independent.

**PHOENIX**

The recovery engine. It keeps a ladder of safe states and a tested rollback path, so the system can step down to a simpler validated mode instead of carrying on with false confidence.

## HOW IT WORKS

1. Before deployment, THEMIS turns each detector's contract into static checks, property tests and small runtime monitors. The Assurance Compiler builds the detector and its verifier along separate paths so that they do not share the same bugs.
2. MIRROR compares the original and compiled versions using semantic probes and meaning-preserving changes, such as renaming a PID or shifting timestamps. It also stores 'semantic checksums' so that any later behaviour change is noticed and traced to one behaviour family.

## RULES IT NEVER BREAKS

- Every production alert has a typed evidence path, or it states openly where inference or visibility is limited.
- 'Not observed' is not the same as 'did not happen': MISSING evidence may become NEGATIVE only when a visibility contract proves the sensors could have seen it.
- Research output never leads directly to a response action. Assurance degrades explicitly when the host is out of envelope and never fails silently.
- Empirical testing is never presented as formal proof. Agreement between detectors is evidence, not proof of truth.

3. At runtime, each conclusion is built as an evidence DAG from kernel observation up to the alert. Types and uncertainty travel with the evidence, and confidence may not grow beyond what the inputs support unless new independent evidence is added.
4. SENTINEL watches cheap invariants all the time. When risk or novelty rises, assurance escalates through levels A0 to A4: from signature checks, to JANUS shadowing, to replay, to offline PARADOX campaigns that search for inputs where detectors disagree and record them in the Failure Atlas.
5. AEGIS fuses the results into the assurance vector. If the host has left the validated envelope, or assurance has decayed over time, it reports ASSURANCE\_DEGRADED or INSUFFICIENT\_EVIDENCE instead of a false BENIGN or MALICIOUS.
6. If a violation continues, PHOENIX moves down the safe-state ladder (P3 Full, P2 Verified, P1 Reflex, P0 Survival) and says which coverage was lost. Every update must already have a tested way back to a known-safe state.

#### WHERE IT RUNS

Split. On the 2 GB endpoint: SENTINEL runtime monitors, typed evidence and provenance state, semantic checksums and a bounded Failure Atlas cache, each with a target of a few MB, plus an optional sampled JANUS shadow of 0-50 MB. Offline on a research machine: the PARADOX, fault-injection, replay and quantization laboratories, which use 0 MB on the endpoint by default.

#### IN THE EXAMPLE ATTACK

When the detector flags the chain (web server spawns a shell, the shell writes and runs /tmp/.x91, that process reads /etc/shadow and connects to 10.0.0.9), Stage 10 makes the alert carry an evidence certificate. The certificate types each link, for example the shadow read as DIRECT and the attack mechanism as INFERRED, and names the smallest witness, such as the /etc/shadow read followed by the outbound connection. If the network sensor was down, the connection is marked MISSING rather than 'did not happen', and the alert's confidence is capped. SENTINEL confirms that the evidence exists and that the host is inside its validated envelope, JANUS may re-check this high-value episode with a different detector type, and the prediction is written to the immutable ledger so that it can be scored against the investigation's outcome later.

Same name, different job: Stage 10's AEGIS (assurance controller) and SENTINEL (runtime monitors) are not the same as Stage 5's AEGIS (response planner) and SENTINEL (the privileged verifier that says PASS or DENY). Stage 10's levels A0-A4 measure how much checking is done. They are not Stage 5's authority classes A0-A5 or Stage 3's certificate levels A0-A5. Stage 10's P0-P3 are safe fallback modes, not Stage 2's compute paths P0-P4.

# 11

## ETERNA

Engines: ETERNA, TEMPUS, LINEAGE, ANCHOR, ATLAS, MORPHEUS, NEXUS, ARK, MNEMOSYNE, HIBERNIA, PALIMPSEST, REQUIEM · Runs across years

Stage 11 keeps PocketSec's security knowledge trustworthy and usable over years, as kernels, sensors, hardware, cryptography and threats change, without losing its evidence trail or safety rules.

### THE PROBLEM IT SOLVES

Technology changes over time. Kernels update, sensors are replaced, compilers and runtimes disappear, and cryptographic algorithms become outdated. A detector that was trusted under old conditions can quietly stop being valid, forget old attacks after retraining, or become impossible to rebuild. Stage 11 manages this aging so that each new version of PocketSec earns its trust again and can be traced, verified and rebuilt.

### THINK OF IT AS

*It is like a national library keeping a key textbook usable for decades: it records every edition's family tree and official seal, re-checks the book when the curriculum changes, keeps only the needed chapters on the desk, and stores a master copy that can be reprinted on any future press.*

### WHAT GOES IN

- Candidate successor artifacts (new or migrated detectors) that have passed Stage 9 and Stage 10 validation
- Changes in the environment: kernel, software, hardware, sensors, workload role, threat population, cryptography and dependencies
- Stage 10 assurance evidence and the Stage 5 authority constitution, which set the limits Stage 11 must respect
- Capsules shared through Stage 7 collective intelligence, which each receiving host validates for itself

### WHAT COMES OUT

- ← Promotion allowed or blocked, based on assurance debt and on whether the successor is legitimate
- ← A signed ancestry record (LINEAGE and ANCHOR) and ARK reconstruction capsules for every production artifact
- ← A small verified active working set of specialists, loaded by HIBERNIA and

handed to AEGIS PRIME for runtime assurance

- ← Fossils and ancestral regression tests that later detector generations must still pass, plus migration plans and risk forecasts

## MAIN PARTS

### ETERNA

A small lifecycle coordinator. It decides whether PocketSec can safely change, move or retire a piece of intelligence, and it keeps assurance debt below a ceiling.

### LINEAGE

Keeps a cryptographic family tree of every artifact (compile, quantize, migrate, merge, retire). It records for each property whether it was PRESERVED, REVALIDATED, WEAKENED, LOST or UNKNOWN in the child.

### ARK

Stores compact preservation capsules, called 'computational DNA': the meaning, contracts, tests, lineage and build recipe needed to rebuild a detector later, even if its original runtime no longer exists.

### TEMPUS

Detects 'epochs', meaning periods in which the kernel, sensors, workload or threats stay stable enough for old test results to still count. It marks when a claim is valid, weakened or invalid in each epoch.

### ANCHOR

Signs and verifies artifacts, lineage steps and capsules. It is crypto-agile, so signature algorithms (including post-quantum ones) can be rotated without breaking the ability to check old history.

### HIBERNIA

Keeps only a small working set of specialist detectors in RAM and leaves the rest signed and dormant on disk. It verifies and activates a specialist when the host or an incident needs it.

## HOW IT WORKS

1. TEMPUS watches for epoch boundaries, such as a kernel upgrade or a new sensor. Each change adds 'assurance debt', because old test results may no longer apply.
2. Any new or migrated detector must be a legitimate successor. It needs constitutional continuity, declared changes in meaning, a valid signed LINEAGE edge, no increase in authority, Stage 10 acceptance and a rollback path. If

## RULES IT NEVER BREAKS

- A descendant cannot silently inherit assurance it has not earned. UNKNOWN blocks inheritance.
- PocketSec may never evolve faster than Stage 10 can assure it: promotion is blocked when assurance debt exceeds the ceiling.
- No Stage 11 mechanism may weaken Stage 10 assurance or escalate authority to make lifecycle management easier.

a property's transfer is UNKNOWN, silent inheritance is blocked.

3. ATLAS maps where each capability is validated, weakened, unknown or invalid across hosts, hardware, kernels and sensors. MORPHEUS uses this map to plan migrations to new sensors or hardware, and NEXUS provides adapters that translate old data meanings into new ones.
4. Promotion is blocked when assurance debt exceeds its ceiling, so PocketSec never evolves faster than Stage 10 can assure it. Independent validation pays the debt back.
5. MNEMOSYNE checks that descendants still detect what ancestors detected, using ancestral probes and historical incident fossils. ARK capsules are rebuilt in clean-room drills by ORPHEUS, because preservation claims expire if they are never tested.
6. PALIMPSEST and JANITOR keep storage bounded through controlled forgetting and deduplication. REQUIEM retires outdated detectors step by step (ACTIVE, DEPRECATED, SHADOW, FOSSILIZED, ARCHIVED) and keeps their lessons as small fossils and tests.

#### IN THE EXAMPLE ATTACK

Stage 11 does not judge this single incident live. The incident context can, however, make HIBERNIA verify (through ANCHOR's signature) and load a deeper credential-theft specialist from the signed cold store into RAM. Afterwards, knowledge from incidents like this is kept as a historical incident fossil and an ancestral probe, so that any future detector generation (after a kernel upgrade, a sensor replacement or a move to ARM hardware) must still catch 'web shell drops a /tmp file that reads /etc/shadow and connects out' before it is promoted. PALIMPSEST keeps the raw telemetry only for a short, bounded time, while the validated mechanism is kept long term.

- Nothing still needed for lineage, reconstruction, assurance or policy may be deleted. Forecasts (CASSANDRA) never retire intelligence on their own, and continuity must work offline.

#### WHERE IT RUNS

Mostly offline, with a small part on the endpoint. On the 2 GB endpoint, one pocketsec process holds the ETERNA/TEMPUS coordinator, the ANCHOR/LINEAGE metadata verifier, the HIBERNIA working-set manager and PALIMPSEST/JANITOR maintenance, each targeted at a few MB. Offline, at build time or in cold storage: MORPHEUS, ARK, MNEMOSYNE, REQUIEM, ATLAS expansion, ORPHEUS, CASSANDRA and CHRONICLE, which use 0 MB on the endpoint by default.

Same name, different job: Stage 11's MNEMOSYNE (capability succession between detector generations) is not Stage 6's MNEMOSYNE (memory maintenance), and Stage 11's ORPHEUS (reconstruction drills) is not Stage 7's ORPHEUS (knowledge sharing between machines).



# Design rules for the whole system

These hold across every stage. Each stage's own rules above are specific cases of them.

---

**Security quality and resource cost are judged together, and the 2 GB machine is a hard limit.**

A detector that catches everything but uses up the machine's memory is useless on small computers. Heavy work may happen offline, but what runs on the machine must stay light.

---

**Raw evidence is kept separate from the compressed version the AI uses, and its history is never lost.**

The AI may simplify events to save space, but an investigator must always be able to go back to the original record.

---

**AI confidence never gives permission to act, and no text can become a privileged command.**

Only a small, fixed list of typed actions exists, and an independent checker (SENTINEL) must approve each one. This stops a confident mistake or injected text from harming the machine.

---

**Knowledge from other machines or from research tools is untrusted until this machine checks it itself, and research tools have no power over the live system.**

A hacked peer, a crowd of fake peers, or a clever but wrong research result must not be able to change a machine's defences.

---

**Every memory, queue and cache on the machine has a fixed maximum size.**

An attacker who floods the machine with events must not be able to make the security system run out of memory and crash.

---

**'Unknown' is a valid answer, and something new is not automatically something bad.**

Guessing creates false alarms or false comfort. New software is often harmless, so the system must say when it is unsure instead of pretending.

---

**Raw events never directly change what the system trusts long-term.**

An attacker could repeat an attack until it looks normal. All new learning waits in quarantine and must pass tests before it is promoted.

---

**Every change must be reversible, and every complex part must beat a simpler method.**

Rollback lets the system recover from a bad update. The baseline test stops parts from surviving just because they sound advanced.

# Glossary

Terms used on this page, with the stage that introduces each one.

## Abstention

The model saying 'I do not know' instead of guessing; this is a valid, expected output.

Stage 0

---

## Assurance envelope

The set of conditions (hardware, kernel, sensors, workload, resources, time) under which a detector was validated. Outside it, results are treated as less trustworthy, not as malicious. Stage 10

---

## Byzantine peer

A participant that may send arbitrary, wrong or malicious information, whether because it is broken or compromised. Stage 7

---

## Catastrophic forgetting

When learning something new makes a model lose an ability it previously had, such as detecting an older attack. Stage 6

---

## Concept drift / behaviour epoch

'Normal' behaviour changing after a legitimate update; an epoch is a period of normality tied to a specific system configuration. Stage 1

---

## Assurance debt

A running total of unverified risk caused by changes such as a new kernel, compiler, sensor or crypto suite. New validation pays it down, and too much debt blocks updates.

Stage 11

---

## Baseline

A simple, well-known method (for example a compact classifier) that any new idea must beat or complement to be kept. Stage 0

---

## Calibration

Whether stated confidence matches reality: when a calibrated model says 90% sure, it is right about 90% of the time. Stage 2

---

## Computational Genome

A structured description of a whole detector design (what state it keeps, what it remembers and forgets, how it learns, how it uses resources) that the search can vary safely, instead of editing raw code. Stage 9

---

## Count-Min Sketch / Bloom filter

Small fixed-size data structures that approximately count or remember what has been seen, trading a little accuracy for bounded memory. Stage 1

---

### **Epistemic type**

A label on each piece of evidence that says how PocketSec knows it: seen directly, derived, inferred, corroborated, confirmed absent, missing or contradicted. Stage 10

---

### **Epoch / concept drift**

A period in which the machine's configuration stays the same; drift is when normal behaviour changes (e.g. after an upgrade), so old predictions may no longer hold. Stage 2

---

### **Falsification**

Actively trying to prove a theory wrong. A theory is kept only if it survives serious attempts to disprove it. Stage 8

---

### **Fossil**

A small, permanent record of a retired detector: its mechanism, probes, minimal witness, failure cases and retirement reason. Future detectors are tested against it so that old lessons are not forgotten. Stage 11

---

### **Hypothesis Genome / Mechanism Grammar**

A structured, typed record of a theory (trigger, state changes, security effect, predicted and forbidden observations), built from a small fixed vocabulary instead of free text. Stage 8

---

### **Intermediate representation**

A standard in-between language that many different inputs are translated into, so later steps only need to understand one format (borrowed from compilers). Stage 1

---

### **Epoch**

A period in which the machine's software and configuration stay the same; a package upgrade or new service starts a new epoch, and old knowledge is kept (possibly dormant) rather than deleted. Stage 6

---

### **Evidence certificate (minimal witness)**

The smallest set of evidence that, found by a bounded search, still produces the same decision. It is attached to the alert so that a person can audit it. Stage 10

---

### **Federated learning**

A way for many machines to train a shared model by sending model updates instead of raw data; Stage 7 treats it only as a baseline to beat. Stage 7

---

### **Homeostatic Runtime**

A controller on the endpoint that keeps memory and CPU below set limits. It moves between cognitive regimes (survival, reflex, adaptive, deep) and reports which coverage was lost. Stage 9

---

### **Informational surprise I(e|s)**

How unexpected an event is given the current state, computed as minus log2 of its probability; rare events score high. Stage 0

---

### **Invariant**

A relationship that stays true even when irrelevant details change, such as the name of the web server program. Stage 3

---

### **Knowledge Fossil**

A permanent, tamper-evident snapshot of a validated knowledge state, identified by its content hash, used for rollback, comparison and bringing old knowledge back. *Stage 6*

---

### **Model slot**

A plug-in point with a fixed input and output format, so the AI model can be replaced without changing the rest of the system.

*Stage 0*

---

### **Oracle / teacher**

A reference used to judge whether a cell is correct; here the DTL model is one oracle and explicit security rules plus labelled data are the other. *Stage 3*

---

### **Phenotype vs computational DNA**

The phenotype is the running executable on a given machine. Computational DNA is the compact description (meaning, contracts, tests, lineage, build recipe) from which a working version can be rebuilt. *Stage 11*

---

### **Provenance / causal ancestry**

The chain of which process caused which action, back to where the activity began.

*Stage 1*

---

### **Quantization (into Behaviour Atoms)**

Rounding a continuous internal state to the nearest item in a fixed set of prototypes, so behaviour can be counted, cached and later compiled. *Stage 2*

---

### **Latent state**

A small internal summary (a list of numbers) that the model keeps to remember what has been happening, instead of storing the full event history. *Stage 2*

---

### **Non-IID data**

Data that differ from machine to machine (a web server behaves differently from a developer laptop), so what is normal on one host may look strange on another. *Stage 7*

---

### **Phenotype / speciation**

A concrete, host-specific configuration built from verified parts. Different 'species' are tuned for different machine roles, such as a web server versus a desktop. *Stage 9*

---

### **Poisoning**

An attack on a learning system in which the attacker feeds it misleading data (for example, repeating a malicious action) so that it learns the wrong thing. *Stage 6*

---

### **Quality-Diversity (MAP-Elites)**

An established search method that keeps the best solution for each of many different niches instead of looking for one overall winner. *Stage 9*

---

### **Representation Tournament**

A competition in which different compiled forms of the same discovery (rule, state machine, small model) are measured on accuracy, false alarms, speed, memory and robustness. The smallest adequate form wins. *Stage 8*

---

## Residual

The part of what was observed that PocketSec's current knowledge cannot explain. A large residual is not automatically malicious. It means the system's current model is not good enough. Stage 8

---

## Shadow execution

Running a new cell silently beside the existing model and logging whether they agree, before the cell is allowed to make real decisions. Stage 3

---

## Validity boundary

The explicit set of situations (states, events, epochs) in which a cell has been tested and is allowed to decide; outside it the cell must not be trusted. Stage 3

---

## ABBREVIATIONS

### ADR

Architecture Decision Record

---

### AION

Architecture Invention and Optimization Nexus

---

### APF

Adaptive Plasticity Field

---

### CEGIS

Counterexample-Guided Inductive Synthesis

---

### CRC

Collective Reproducibility Capsule

---

## Safe-state lattice

PocketSec's ladder of fallback modes (P3 Full, P2 Verified, P1 Reflex, P0 Survival). Each step down gives up some detection capability in return for more reliability.

Stage 10

---

## Sybil attack

An attacker creates many fake identities so that one source looks like many independent voices. Stage 7

---

### AICT

Adaptive Intelligence Crystallization Theory

---

### AOP

Adaptive Observation Policy

---

### CBF

Causal Belief Field

---

### CF

counterfactual result

---

### CSG

Collective Security Gain

---

**DCR**

Discovery Compression Ratio

---

**DP**

Differential Privacy

---

**ECE**

Expected Calibration Error

---

**EE**

Epistemic Efficiency

---

**EXT**

external knowledge mapping

---

**INF**

model inference

---

**JIT**

just-in-time (compilation)

---

**LUCID**

Latent Uncertainty and Causal Identification Dynamics

---

**MIR**

Mechanism Intermediate Representation

---

**MSSC**

Minimum Sufficient Security Computation

---

**NKA**

Negative Knowledge Atom

---

**PCB**

PocketSec Cell Bytecode

---

**DER**

deterministic derivation

---

**DTL**

Dynamic Transition Lattice

---

**ECHO**

Evidence-Causal Heterogeneity-aware Orchestration

---

**EIG**

Expected Information Gain

---

**FCK**

Falsification-Carrying Knowledge

---

**IPB**

Information-Per-Byte

---

**KCG**

Knowledge Conservation Gate

---

**MDL**

Minimum Description Length

---

**MSDL**

Minimum Security Description Length

---

**NERA**

Novelty-Energy Relational Automaton

---

**OBS**

direct observation

---

**PCK**

Proof-Carrying Knowledge

---

**QD**Quality-Diversity

---

**SKA**Security Knowledge Atom

---

**SSIR**Security Semantic Intermediate  
Representation

---

**VOI**Value of Information

---

**R\_K**Poisoning Propagation Number

---

**SL**Sovereignty Loss

---

**UNK**unknown/unobserved

---

---

PocketSec is a research prototype. This guide is written from its thirteen architecture documents ([docs/architecture/sources/](#)) and describes the design; it does not report experimental results.